گزارش فنی مسیر تحلیل و پیاده‌سازی پروژه

Stack Overflow Churn Prediction

فصل ۳ — سیستم تفسیری Rule-based

۳.۱ مقدمه

این فصل به توسعه‌ی سیستم تفسیری Rule-based برای پیش‌بینی Churn می‌پردازد. هدف این سیستم:

1. **قابل توضیح بودن (Interpretability):** هر پیش‌بینی باید قابل توضیح برای ذی‌نفعان غیرفنی باشد
2. **پارامترهای قابل تنظیم:** T_p و A_p برای هر پرسونا جداگانه تعیین شوند
3. **مبنایی برای مقایسه:** خط‌مشی (Baseline) برای مدل‌های پیچیده‌تر

۳.۲ فرمول‌بندی قاعده

۳.۲.۱ قاعده‌ی اصلی

برای کاربر u در پرسونا p:

$$\boxed{\text{Churn}(u) = \mathbb{1}\left[WA(u, T_p) < A_p\right]}$$

که در آن:

پارامتر	تعریف
T_p	پنجره‌ی زمانی مشاهده (Time Period)
A_p	آستانه‌ی فعالیت (Activity Threshold)
WA	فعالیت وزن‌دار (Weighted Activity)
$\mathbb{1}[\cdot]$	تابع نشانگر (اگر شرط برقرار باشد ۱، وگرنه ۰)

به زبان ساده: اگر مجموع فعالیت وزن‌دار کاربر در بازه‌ی T_p کمتر از آستانه‌ی A_p باشد، کاربر «ریزش» پیش‌بینی می‌شود.

۳.۲.۲ محاسبه‌ی Weighted Activity

$$WA(u, T_p) = \sum_{e \in \text{Events}(u, T_p)} \text{gain}(e)$$

که در آن:

- $\text{Events}(u, T_p)$: رویدادهای کاربر u در پنجره‌ی T_p
- $\text{gain}(e)$: وزن رویداد (از XGBoost Feature Importance)

۳.۲.۳ چرا Gain به‌جای Count؟

دو رویکرد مقایسه شد:

رویکرد	فرمول	مزیت	معایب
Count-based	$\sum \text{count}(e)$	ساده	همه‌ی اکشن‌ها برابر وزن دارند
Gain-based	$\sum \text{gain}(e)$	وزن‌دهی هوشمند	وابسته به XGBoost

> 💡 **نکته مفهومی:**
>
> اکشن‌های مختلف اهمیت متفاوتی دارند. در Stack Overflow، یک پاسخ (Answer) ارزشمندتر از یک نظر (Comment) است. Gain از XGBoost این تمایز را به‌صورت داده‌محور یاد می‌گیرد.

۳.۳ استخراج Gain با XGBoost

۳.۳.۱ هدف

می‌خواهیم بفهمیم **هر نوع فعالیت برای هر پرسونا چقدر مهم است**. آیا نوشتن Answer مهم‌تر است یا دادن Comment؟ آیا این برای همه‌ی پرسوناها یکسان است؟

۳.۳.۲ روش One-vs-Rest

برای هر پرسونا، یک **مدل دسته‌بندی باینری** می‌سازیم:

پرسونای ۳ در مقابل بقیه:

- کلاس ۱: کاربران پرسونای ۳
- کلاس ۰: تمام کاربران دیگر

این کار را برای هر ۲۰ پرسونا تکرار می‌کنیم.

۳.۳.۳ چرا XGBoost؟

متد XGBoost یک الگوریتم یادگیری ماشین قدرتمند است که:

1. **درخت‌های تصمیم** را به صورت متوالی می‌سازد
2. هر درخت سعی می‌کند **خطای درخت‌های قبلی** را اصلاح کند
3. در نهایت، **اهمیت هر ویژگی** را محاسبه می‌کند

به زبان ساده‌تر: XGBoost مثل یک کارآگاه است که سوالات متوالی می‌پرسد:

- «آیا این کاربر پاسخ زیادی داده؟» ← اگر بله، احتمالاً پرسونای X
- «آیا ویرایش زیادی کرده؟» ← اگر بله، احتمالاً پرسونای Y

هر چه یک فعالیت (ویژگی) در تشخیص پرسونا **مؤثرتر** باشد، **وزن بالاتری** می‌گیرد.

۳.۳.۴ کد محاسبه Gain

فایل `feature_importance_by_xgboost.py` :

```python
import polars as pl
import xgboost as xgb
import pandas as pd
import re

# بارگذاری ماتریس ویژگی‌ها
fm = pl.read_parquet("feature_matrix_v2.parquet")

# نگاشت ActionTypeId به نام
action_df = pd.read_csv("ActionTypes.csv")
ACTION_MAP = dict(zip(action_df["ActionTypeId"], action_df["Name"]))

# ستون‌های ویژگی
drop_cols = ["user_id", "AccountCreationDate", "LastAccessDate"]
feature_cols = [c for c in fm.columns if c not in drop_cols + ["persona"]]

personas = fm.select("persona").unique().to_series().to_list()
results = []

# آموزش one-vs-rest برای هر پرسونا
for P in personas:
    print(f"=== Training for persona {P} ===")

    # تعریف هدف باینری
    df = fm.with_columns([
        (pl.col("persona") == P).cast(pl.Int8).alias("target")
    ])

    X = df.select(feature_cols).to_pandas()
    y = df.select("target").to_pandas()["target"]

    # آموزش XGBoost
    dtrain = xgb.DMatrix(X, label=y)
    params = {
        "objective": "binary:logistic",
        "eval_metric": "logloss",
        "max_depth": 6,
        "learning_rate": 0.1,
    }
    model = xgb.train(params, dtrain, num_boost_round=200)

    # استخراج اهمیت ویژگی‌ها
    importance = model.get_score(importance_type="gain")

    # ساخت dataframe با نام اکشن‌ها
    rows = []
    for col in feature_cols:
        gain_value = importance.get(col, 0)
        match = re.match(r"count_action_(\d+)", col)
```

```
        if match:
            action_id = int(match.group(1))
            action_name = ACTION_MAP.get(action_id, "Unknown_Action")
        else:
            action_id = None
            action_name = col

        rows.append({
            "persona": P,
            "feature": action_name,
            "action_type_id": action_id,
            "gain": gain_value
        })

    imp_df = pd.DataFrame(rows)

    # نرمال‌سازی
    imp_df["gain"] = imp_df["gain"] / (imp_df["gain"].sum() + 1e-12)
    results.append(imp_df)

# ذخیره
final_df = pd.concat(results)
final_df = final_df.sort_values(["persona", "gain"], ascending=[True,
False])
final_df.to_csv("core_activity_per_persona_named.csv", index=False)
```

۳.۳.۵ نتیجه: وزن‌های هر پرسونا

فایل `core_activity_per_persona_named.csv` حاوی وزن‌های محاسبه‌شده است:

gain	action_type_id	feature	persona
0.161	2	Create_Answer	0
0.092	1	Create_Question	0
0.059	14	Edit_Body	0
0.172	70	Create_Comment	1
0.098	2	Create_Answer	1
0.311	2	Create_Answer	3

۳.۳.۶ تفسیر نتایج

پرسونا	مهم‌ترین فعالیت	Gain	تفسیر
0	Create_Answer	0.16	این پرسونا بیشتر پاسخ‌دهنده است
1	Create_Comment	0.17	این پرسونا بیشتر نظردهنده است
3	Create_Answer	0.31	این پرسونا **خیلی** پاسخ‌دهنده است!

> 💡 **مثال مفهومی:**
>
> پرسونای ۳ احتمالاً شامل کاربرانی مثل Jon Skeet است که اصلاً سؤال نمی‌پرسند و فقط پاسخ می‌دهند. برای این پرسونا، کاهش تعداد Answer خیلی نگران‌کننده‌تر از کاهش Comment است.

۳.۴ تخمین همزمان T_p و A_p

۳.۴.۱ چالش اصلی

دو پارامتر T_p و A_p **به هم وابسته** هستند:

- اگر T_p بزرگ‌تر شود، WA بیشتر می‌شود $\rightarrow$ A_p هم باید تغییر کند
- بنابراین نمی‌توانیم هر کدام را جداگانه بهینه کنیم

راه‌حل: جستجوی همزمان بهترین ترکیب (T_p, A_p)

۳.۴.۲ تعریف T_p بر اساس P95 Gap

$$T_p = \alpha \times \text{p95_gap}_p$$

که در آن:

- p95_gap_p: صدک ۹۵ فاصله‌ی زمانی بین اکشن‌های کلیدی پرسونا
- α: ضریب تنظیم

چرا صدک ۹۵؟ می‌خواهیم ۹۵٪ از فاصله‌های طبیعی را پوشش دهیم.

```
# برای هر پرسونا p95_gap محاسبه
gap_stats = (
    elog
    .filter(pl.col("gap_time_key_actions_seconds") > 0)
    .group_by("persona")
    .agg(
        pl.col("gap_time_key_actions_seconds")
        .quantile(0.95)
        .alias("p95_gap")
    )
)
p95_map = dict(zip(gap_stats["persona"], gap_stats["p95_gap"]))
```

۳.۴.۳ روش‌های تست‌شده

سه روش مختلف برای یافتن همزمان (T_p, A_p) پیاده‌سازی و مقایسه شد:

روش	فایل	ایده‌ی اصلی
Grid Search + Logistic Regression	`tp_ap_gridsearch.py`	استخراج Ap از Decision Boundary
Scalar WA + Logistic Regression	`tp_ap_logistic.py`	رگرسیون روی یک ویژگی (WA)
Grid Search + XGBoost	`tp_ap_xgboost.py`	یافتن Ap با Precision-Recall Curve

۳.۵ روش ۱: Grid Search + Logistic Regression

۳.۵.۱ ایده

از Decision Boundary رگرسیون لجستیک برای استخراج A_p استفاده می‌کنیم.

۳.۵.۲ مراحل

فایل `tp_ap_gridsearch.py` :

```
# کاندیداهای alpha
alpha_values = np.arange(1, 16, 4).tolist()  # [1, 5, 9, 13]

for persona, p95_gap in p95_map.items():
    elog_p = train_elog.filter(pl.col("persona") == persona)
    max_ts = elog_p.select(pl.max("timestamp")).item()

    for alpha in alpha_values:
        Tp_sec = float(alpha * p95_gap)
        window_start = max_ts - pl.duration(seconds=int(Tp_sec))
        df_w = elog_p.filter(pl.col("timestamp") >= window_start)

        # ساخت ماتریس ویژگی: sum(gain) per action_type
        feat = (
            df_w
            .group_by(["user_id", "action_type_id"])
            .agg(pl.col("gain").sum().alias("gain"))
            .pivot(values="gain", index="user_id", on="action_type_id")
            .fill_null(0)
        )

        # آموزش Logistic Regression
        model = LogisticRegression(max_iter=2000)
        model.fit(X, y)

        coef = model.coef_[0]
        intercept = model.intercept_[0]

        # استخراج Ap از Decision Boundary
        # در مرز تصمیم: intercept + sum(w_j * x_j) = 0
        # => Ap = -intercept
        Ap = float(-intercept)
```

۳.۵.۳ مشکل این روش

- A_p مستقیماً از intercept استخراج می‌شود که **تفسیر دقیقی ندارد**
- وابسته به مقیاس ویژگی‌ها است
- نمی‌تواند روابط غیرخطی را بگیرد

۳.۶ روش ۲: Scalar WA + Logistic Regression

۳.۶.۱ ایده

به‌جای ماتریس ویژگی، فقط از **یک ویژگی (WA)** استفاده می‌کنیم.

۳.۶.۲ مراحل

فایل `tp_ap_logistic.py`:

```
for persona, p95_gap_sec in p95_map.items():
    for alpha in ALPHAS:  # [3.0, 5.0, 7.0, 10.0]
        Tp_sec = alpha * p95_gap_sec
        window_start = train_cutoff - pl.duration(seconds=int(Tp_sec))

        # فیلتر رویدادها در پنجره
        win = core_p.filter(
            (pl.col("timestamp") >= window_start) &
            (pl.col("timestamp") <= train_cutoff)
        )

        # محاسبه WA = sum(gain)
        wa = (
            win
            .group_by("user_id")
            .agg(pl.col("gain").sum().alias("weighted_activity"))
        )

        # آموزش Logistic Regression با یک ویژگی
        X = y_df["weighted_activity"].to_pandas().values.reshape(-1, 1)
        y = y_df["churn_seed"].to_pandas().values

        model = LogisticRegression(max_iter=2000)
        model.fit(X, y)

        coef = float(model.coef_[0][0])
        intercept = float(model.intercept_[0])

        # Ap = نقطه‌ی Decision Boundary در فضای WA
        Ap = -intercept / coef
```

۳.۶.۳ مزیت و مشکل

✅ **مزیت:** A_p مستقیماً در فضای WA قابل تفسیر است

❌ **مشکل:** رگرسیون لجستیک خطی است و نمی‌تواند الگوهای پیچیده را بگیرد

۳.۷ روش ۳: Grid Search + XGBoost (روش نهایی)

۳.۷.۱ ایده

از XGBoost برای پیش‌بینی احتمال Churn استفاده می‌کنیم، سپس بهترین A_p را با **Precision-Recall Curve** پیدا می‌کنیم.

۳.۷.۲ چرا XGBoost؟

- **غیرخطی:** روابط پیچیده‌تر را می‌گیرد
- **مقاوم:** نسبت به نویز مقاوم‌تر است
- **خروجی احتمال:** احتمال Churn را می‌دهد (نه فقط ۰/۱)

۳.۷.۳ مراحل

فایل `tp_ap_xgboost.py` :

```
alpha_values = np.arange(3, 10, 1).tolist()
grace_period_sec = 90 * 86400  # 90 روز

for persona, p95_gap in p95_map.items():
    elog_p = elog.filter(pl.col("persona") == persona)

    for alpha in alpha_values:
        Tp_sec = float(alpha * p95_gap)

        for year, grace_coef in cutoff_years:
            cutoff = pl.datetime(year, 1, 1)
            window_start = cutoff - pl.duration(seconds=int(Tp_sec))

            # رویدادهای در پنجره
            win = elog_p.filter(
                (pl.col("timestamp") >= window_start) &
                (pl.col("timestamp") <= cutoff)
            )

            # محاسبه WA
            wa = (
                win
                .group_by("user_id")
                .agg(pl.col("gain").sum().alias("WA"))
            )

            # برچسب واقعی: فعالیت بعد از cutoff + grace
            future = (
                elog_p
                .filter(
                    (pl.col("timestamp") > cutoff +
pl.duration(seconds=grace_period_sec)) &
                    (pl.col("timestamp") < cutoff + pl.duration(seconds=366
* 86400 + grace_period_sec))
                )
                .group_by("user_id")
                .agg(pl.len().alias("cnt"))
            )

            actual = (
                wa.join(future, on="user_id", how="left")
                  .with_columns([

(pl.col("cnt").is_null()).cast(pl.Int8).alias("actual_churn")
                  ])
            )

            X = pdf["WA"].values.reshape(-1, 1)
            y = pdf["actual_churn"].values
```

```
            # آموزش XGBoost
            model = XGBClassifier(
                max_depth=3,
                learning_rate=0.05,
                n_estimators=60,
                subsample=0.9,
                eval_metric="logloss",
            )
            model.fit(X, y)

            pred_prob = model.predict_proba(X)[:, 1]

            # یافتن Ap با Precision-Recall Curve
            precision, recall, thresholds = precision_recall_curve(y,
pred_prob)

            target_recall = 0.8
            target_precision = 0.65

            # یافتن threshold که هر دو هدف را برآورده کند
            cond = (recall[1:] >= target_recall) & (precision[1:] >=
target_precision)
            idxs = np.where(cond)[0]
            if idxs.size:
                Ap = float(thresholds[idxs[0]])
            else:
                # Fallback: مناسب recall با precision بالاترین
                cand = np.where(recall[1:] >= target_recall)[0]
                if cand.size:
                    best = cand[np.argmax(precision[1:][cand])]
                    Ap = float(thresholds[best])
```

۳.۷.۴ Calibration

برای اطمینان از تطابق نرخ پیش‌بینی با نرخ واقعی:

```
pred = (pred_prob >= Ap).astype(int)
pred_rate = float(pred.mean())

# کالیبراسیون
Ap_Calibrated = Ap * (y.mean() / pred_rate) if pred_rate > 0 else Ap
```

۳.۷.۵ چرا این روش انتخاب شد؟

معیار	Logistic	XGBoost
غیرخطی	❌	✅

معیار	Logistic	XGBoost
مقاوم به نویز	❌	✅
کنترل Precision/Recall	❌	✅
کالیبراسیون	سخت	آسان

۳.۸ ارزیابی: Sliding Window

۳.۸.۱ مشکل Single-point Evaluation

اگر فقط روی یک تاریخ ارزیابی کنیم:

- نتایج به آن تاریخ خاص وابسته می‌شوند
- عملکرد واقعی مدل مشخص نمی‌شود

۳.۸.۲ روش Sliding Window

به‌جای یک تاریخ، روی **پنجره‌های متعدد** ارزیابی می‌کنیم.

فایل `pragmatic_real_eval.py`:

```
# تولید پنجره‌های ۶ ماهه از ۲۰۱۵ تا ۲۰۲۳
def generate_windows(start_year=2015, end_year=2023, step_months=6):
    windows = []
    year = start_year
    month = 1

    while year < end_year or (year == end_year and month == 1):
        cutoff = datetime(year, month, 1)
        windows.append(cutoff)

        month += step_months
        if month > 12:
            month -= 12
            year += 1

    return windows

windows = generate_windows(2012, 2024, 6)  # ۲۴ پنجره
```

۳.۸.۳ محاسبه‌ی Median F1

برای هر ترکیب $(T_p, A_p, \text{persona})$:

1.معیار F1 روی هر پنجره محاسبه شود

1. **میانه‌ی F1** روی همه‌ی پنجره‌ها گزارش شود

```
window_f1_scores = []

for cutoff_dt in windows:
    # ... محاسبه‌ی F1 روی این پنجره
    window_f1_scores.append(f1)

median_f1 = np.median(window_f1_scores)
```

چرا میانه؟ مقاوم به Outlier است. اگر یک پنجره نتایج عجیب داشته باشد، میانه تحت تأثیر قرار نمی‌گیرد.

۳.۹ تعریف Actual Churn

۳.۹.۱ Grace Period

بعد از پایان پنجره‌ی مشاهده، یک **دوره‌ی مهلت (Grace Period)** در نظر گرفته می‌شود:

```
grace_days = 180
grace_sec = grace_days * 86400
```

۳.۹.۲ منطق تعیین Actual Churn

```
# رویدادهای بعد از cutoff + grace
after = (
    elog
    .filter(pl.col("persona") == persona)
    .filter(
        (pl.col("timestamp") > cutoff + pl.duration(seconds=grace_sec))
    )
    .group_by("user_id")
    .agg(pl.len().alias("cnt_after"))
)

# اگر هیچ رویدادی بعد از grace نداشت → churn = 1
eval_df = (
    wa.join(after, on="user_id", how="left")
      .with_columns([

(pl.col("cnt_after").is_null()).cast(pl.Int8).alias("actual_churn")
      ])
)
```

۳.۹.۳ تایم‌لاین ارزیابی

```
|<---- Tp ---->|<-- Grace -->|<---- Observation ---->|
|              |             |                       |
window_start   cutoff        cutoff+grace            cutoff+grace+365d
```

۳.۱۰ محاسبه‌ی WA و پیش‌بینی

۳.۱۰.۱ WA ساده

```
wa = (
    win
    .group_by("user_id")
    .agg(pl.col("gain").sum().alias("WA"))
)

# پیش‌بینی
wa = wa.with_columns([
    (pl.col("WA") < Ap).cast(pl.Int8).alias("pred_churn")
])
```

۳.۱۰.۲ معیارهای ارزیابی

```
# Accuracy
accuracy = float((y_true == y_pred).mean())

# Precision
TP = np.sum((y_true == 1) & (y_pred == 1))
FP = np.sum((y_true == 0) & (y_pred == 1))
precision = TP / (TP + FP) if (TP + FP) > 0 else 0.0

# Recall
FN = np.sum((y_true == 1) & (y_pred == 0))
recall = TP / (TP + FN) if (TP + FN) > 0 else 0.0

# F1
f1 = 2 * (precision * recall) / (precision + recall)
```

توضیح معیارها:

معیار	معنی	اهمیت در Churn
Precision	از پیش‌بینی‌های churn، چند درصد درست بود؟	اگر کم باشد، منابع هدر می‌رود
Recall	از churnهای واقعی، چند درصد را یافتیم؟	اگر کم باشد، کاربران را از دست می‌دهیم

معیار	معنی	اهمیت در Churn
F1	میانگین هارمونیک Precision و Recall	تعادل بین دو معیار

۳.۱۱ نتایج سیستم Rule-based

۳.۱۱.۱ بهترین پارامترها per-persona

از فایل `best_params_validation.csv` :

پرسونا	α	Tp (روز)	Ap	Val F1	Val Recall
0	4.5	67	0.029	0.30	0.85
1	4.5	201	0.070	0.51	0.78
3	4.5	19	0.014	0.21	0.85
9	4.5	447	0.167	0.63	0.88
18	3.5	1435	0.217	0.84	1.00

۳.۱۱.۲ مشاهدات کلیدی

1. **تفاوت بین پرسوناها:** T_p از ۱۹ روز (پرسونای ۳) تا ۱۴۳۵ روز (پرسونای ۱۸) متغیر است
2. **Recall بالا:** اکثر پرسوناها دارند (یعنی اکثر ریزش‌ها شناسایی می‌شوند) Recall > 0.80
3. **F1 متغیر:** از ۰.۲۱ تا ۰.۸۴

۳.۱۲ محدودیت‌های شناسایی شده

۳.۱۲.۱ Temporal Drift

پارامترهای استخراج‌شده در بازه‌های زمانی اخیر با دقت بالاتری نسبت به گذشته عمل می‌کنند.

Persona	Median F1	Std F1
0	0.30	0.08
2	0.39	0.15
3	0.21	0.11

مشاهده: Std F1 بالا نشان‌دهنده‌ی حساسیت به Temporal Drift است.

۳.۱۲.۲ Static Parameters

پارامترهای ثابت نمی‌توانند تغییرات بلندمدت را پوشش دهند.

ساده WA ۳.۱۲.۳

فرمول WA ساده:

- **Trend** را نمی‌بیند (آیا فعالیت رو به افزایش یا کاهش است؟)
- **Recency** را نمی‌بیند (آخرین فعالیت چه زمانی بود؟)

> 💡 **مثال Stack Overflow:**
>
> دو کاربر با WA = 0.6:
>
> - کاربر A: فعالیت یکنواخت در ۱۹ روز ← احتمالاً فعال می‌ماند
> - کاربر B: همه فعالیت در روز اول، سپس سکوت ← احتمالاً ریزش می‌کند
>
> قانون ساده هر دو را **یکسان** می‌بیند!

۳.۱۳ خلاصه‌ی الگوریتم

```
Algorithm: Rule-based Churn Prediction
──────────────────────────────────────

Input: core_eventlog, persona_id, cutoff_date
Output: list of (user_id, pred_churn)

1. Load Tp, Ap for this persona
2. Define window: [cutoff − Tp, cutoff]
3. For each user active in window:
      WA = sum(gain) of core events
      pred_churn = 1 if WA < Ap else 0
4. Return predictions
```

۳.۱۴ فایل‌های مرتبط

فایل	نقش
`feature_importance_by_xgboost.py`	استخراج Gain با XGBoost
`core_activity_per_persona_named.csv`	وزن‌های هر فعالیت per پرسونا
`tp_ap_gridsearch.py`	روش ۱: Grid Search + Logistic Regression
`tp_ap_logistic.py`	روش ۲: Scalar WA + Logistic Regression
`tp_ap_xgboost.py`	روش ۳: Grid Search + XGBoost (نهایی)
`pragmatic_real_eval.py`	ارزیابی Sliding Window
`best_params_validation.csv`	بهترین پارامترها

۳.۱۵ خلاصه‌ی فصل

در این فصل:

- ✅ قاعده‌ی تفسیری $WA(T_p) < A_p$ فرمول‌بندی شد
- ✅ روش استخراج Gain با XGBoost توضیح داده شد
- ✅ سه روش برای تخمین همزمان (T_p, A_p) مقایسه شد:
  - Grid Search + Logistic Regression
  - Scalar WA + Logistic Regression
  - **Grid Search + XGBoost** (روش نهایی)
- ✅ ارزیابی Sliding Window برای پایداری نتایج
- ✅ محدودیت‌ها شناسایی شد: Temporal Drift، عدم توجه به Trend و Recency

خروجی اصلی:

- `core_activity_per_persona_named.csv` (وزن‌ها)
- `best_params_validation.csv` (پارامترها)
- `tp_ap_xgb_joint_search_sliding.csv` (نتایج Grid Search)

نتیجه‌گیری: سیستم Rule-based عملکرد قابل قبولی دارد (Recall بالا) اما برای مقابله با Temporal Drift و بهبود دقت، نیاز به بهبود فرمول WA است که در فصل بعد بررسی می‌شود.